

Cloud Computing on **AWS**



From racking servers to renting superpowers —
the fundamentals, then the eight services you'll actually use.

 Cloud 101

 EC2

 IAM

 S3

 ECR

 EKS

 Telemetry

 Lambda

Today's route

1

Cloud fundamentals

Cloud-agnostic ideas that transfer to any provider



2

EC2

Virtual machines — compute for rent



3

IAM

Who can do what — identity & permissions



4

S3

Object storage — infinitely deep bucket



5

ECR

A home for your container images



6

EKS

Kubernetes without babysitting the control plane



7

Telemetry

Metrics, logs & traces with CloudWatch + X-Ray



8

Lambda

Run code without any server at all



PART 1

Cloud Fundamentals

Provider-agnostic ideas: what the cloud actually is, how it's sold, and who's responsible for what.



2005: you want to launch a website



Buy servers

Spec CPUs, RAM, disks.
Wait 4–8 weeks for delivery.



Rent rack space

Data center contract, power,
cooling, physical access badges.



Wire everything

Switches, firewalls, cables,
public IP blocks from an ISP.



Hire ops staff

Someone drives to the DC
at 3 AM when a disk dies.



Months of lead time and a huge upfront bill — before serving a single user. Guess capacity wrong and you either melt under load or pay for idle metal.

CapEx-heavy · slow to scale · you own every failure

The cloud exists because every company was solving this same problem, badly, in parallel.

What “the cloud” actually means



Someone else's computers, rented by the hour, self-service over the internet. NIST defines five essential characteristics — if a service lacks one, it's just hosting.



On-demand self-service

Provision via console or API — no phone call, no ticket, no human.



Broad network access

Reachable over standard networks from anywhere, any device.



Resource pooling

Provider's hardware is shared by many tenants — you rent slices.



Rapid elasticity

Scale from 1 to 1,000 machines in minutes, and back down.



Measured service

Metered like electricity — pay only for what you use.

Service models: how much do you want to manage?

	On-prem	IaaS	PaaS / CaaS	FaaS	SaaS
App code	You	You	You	You	Provider
Runtime	You	You	You	Provider	Provider
Containers / OS	You	You	Provider	Provider	Provider
Virtual machine	You	You	Provider	Provider	Provider
Physical servers	You	Provider	Provider	Provider	Provider
Networking & DC	You	Provider	Provider	Provider	Provider
	Your rack	EC2	EKS · Beanstalk	Lambda	Gmail

Moving right: less to manage, less to control, faster to ship.

Deployment models: where does it run?



Public cloud

Shared provider infrastructure — AWS, Azure, GCP. Fastest start, global reach, pure OpEx.

Most startups & this bootcamp



Private cloud

Cloud-style self-service on hardware only you use — on-prem OpenStack or dedicated hosts.

Banks, telcos, governments



Hybrid

Private + public working together. Sensitive data stays home; burst workloads go public.

Regulated firms modernizing



Multi-cloud

Two or more public providers — for leverage, resilience, or acquisitions. Costs real complexity.

Large enterprises



Trap to avoid: “multi-cloud for portability” sounds strategic but often means paying the complexity tax of two providers while mastering neither. Pick one, go deep.

The shared responsibility model

AWS — security OF the cloud

- Physical data centers, guards, cameras
- Hardware, networking, host hypervisor
- Managed-service software (S3, Lambda runtime)
- Global infrastructure: Regions & AZs

*“Is the building on fire? Is the hypervisor patched?”
— AWS's problem.*

You — security IN the cloud

- Your data — encryption, classification, backups
- IAM: who can access what (Part 3!)
- OS patching on your EC2 instances
- Security-group / firewall rules you write
- Your application code & its dependencies

*“Is the S3 bucket public? Is root MFA on?”
— your problem.*

Almost every famous “cloud breach” was a customer-side mistake — a public bucket, a leaked key, an open port. The cloud was fine.

Regions, Availability Zones & edge

Region · ap-south-1 (Mumbai)

AZ a

1+ data centers
Own power, cooling,
network



AZ b

1+ data centers
Own power, cooling,
network



AZ c

1+ data centers
Own power, cooling,
network



Region

A geographic area with 3+ isolated AZs. You pick one per workload; data doesn't leave unless you move it.

Availability Zone

Independent failure domain — separate power & flooding risk, linked by fast private fiber. Spread across 2+ AZs = high availability.

Edge location

400+ mini-sites (CloudFront CDN) that cache content close to users — including Kathmandu's ISPs' upstream hubs.



For Nepal: ap-south-1 (Mumbai) is the lowest-latency region — typically ~40–70 ms from Kathmandu. Choose Regions by latency, price, and data-residency law.

The economics: CapEx → OpEx

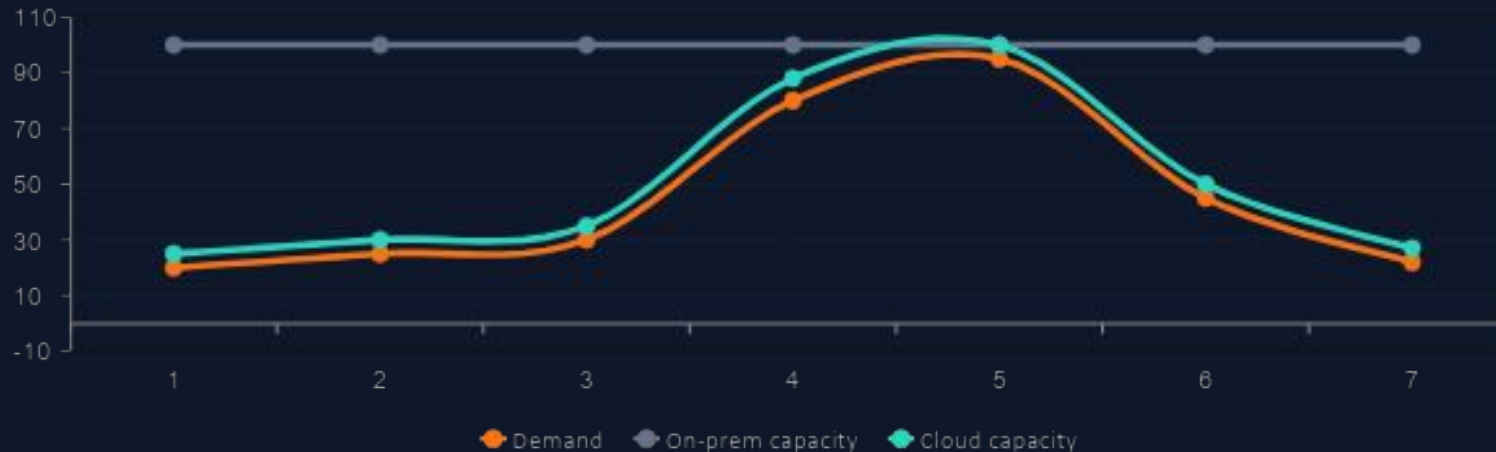
CapEx — own it

Big upfront purchase · depreciates over years · sized for peak load · idle 80% of the time · upgrades = new purchase

OpEx — rent it

Pay per hour / per GB / per request · scales with actual demand · zero upfront · stop paying when you stop using

Paying for capacity vs paying for demand



The gap is money

Grey – orange = capacity you bought but didn't use.

Teal hugs the orange line: elasticity means capacity tracks demand — you pay for the area under teal, not the grey rectangle.

PART 2

EC2

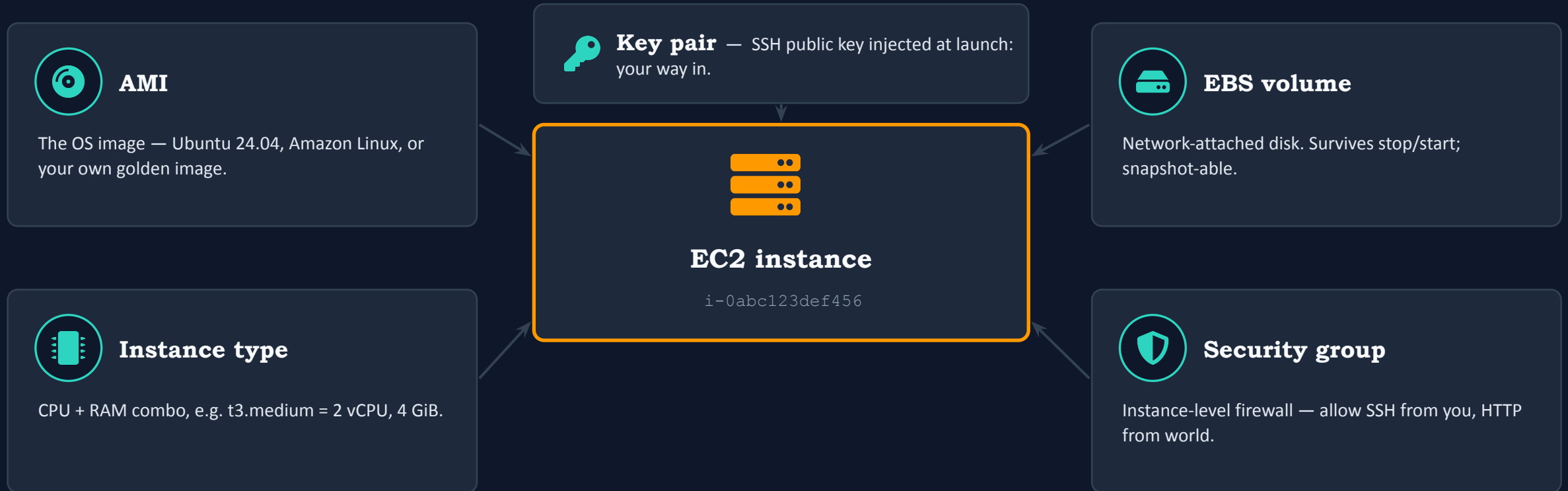
Elastic Compute Cloud — virtual machines by the hour.
The service every other service secretly runs on.



Anatomy of an EC2 instance



You need a Linux box: some CPU & RAM, a disk, an OS, a firewall, and a way to SSH in. EC2 splits that wish into five named parts.



Same ideas as Vagrant: box → AMI · Vagrantfile CPU/RAM → instance type · synced disk → EBS · forwarded ports → security group.

Decoding instance types

t3.medium

family — workload shape (t = burstable general purpose)

generation — hardware version, newer = faster & cheaper

size — CPU/RAM scale: medium → large → xlarge doubles each step

t / m

General purpose

Web servers, dev boxes, small DBs — our labs

c

Compute optimized

CI runners, batch jobs, video encoding

r / x

Memory optimized

Redis, big JVM heaps, in-memory analytics

g / p

GPU accelerated

ML training & inference

i / d

Storage optimized

High-IOPS databases, search clusters

Lifecycle & the three ways to pay



stopped = no compute billing, EBS still billed, can restart. **terminated** = gone forever.



On-Demand

Pay per second, cancel anytime.

1×

Baseline · dev · unpredictable load



Savings Plans / Reserved

Commit to 1–3 years of usage.

≈ 0.4–0.6×

Steady production workloads



Spot

Bid on spare capacity — can be reclaimed with a 2-min warning.

≈ 0.1–0.3×

CI runners, batch, stateless K8s nodes



Real fleets mix all three: reserved for the floor, on-demand for the wiggle, spot for anything that can die and retry. EKS node groups (Part 6) make this easy.

Launching one from the CLI

```
● ● ● bash - aws-cli

$ aws ec2 run-instances \
  --image-id ami-0f5ee92e2d63afc18 \
  --instance-type t3.micro \
  --key-name bootcamp-key \
  --security-group-ids sg-0a1b2c3d \
  --tag-specifications 'ResourceType=instance,
    Tags=[{Key=Name,Value=lab-web-01}]'

$ aws ec2 describe-instances \
  --filters Name=tag:Name,Values=lab-web-01 \
  --query 'Reservations[].Instances[].PublicIpAddress'
[ "13.235.67.89" ]

$ ssh -i bootcamp-key.pem ubuntu@13.235.67.89
ubuntu@lab-web-01:~$ █
```

1 run-instances

One API call replaces the 2005 slide: hardware, OS, network, firewall — provisioned in ~40 s.

2 describe + --query

Everything is queryable JSON. JMESPath filters — jq energy, built in.

3 ssh in

Your key pair from launch. From here it's the Linux module all over again.



Remember to terminate: `aws ec2 terminate-instances --instance-ids i-...` — the free tier forgives, your credit card doesn't.

PART 3

IAM

Identity & Access Management — who (or what) may do which action on which resource. The service that guards all the others.



Four primitives rule everything



Day 1 temptation: everyone shares the root account password. One leaked laptop later, your account mines crypto in three regions. IAM exists so root can retire.



User

A person (or legacy app) with long-term credentials — password + access keys.

```
sagyam@bootcamp
```



Group

A bag of users. Attach policies to groups, never to individual users.

```
developers, admins
```



Role

An identity with NO password that services or users temporarily assume. Credentials auto-rotate.

```
eks-node-role
```



Policy

A JSON document that says Allow/Deny + Action + Resource. Attach to any of the three.

```
S3ReadOnly
```



Mental model: users & roles are WHO, policies are WHAT THEY MAY DO, groups are just bulk assignment. Everything you'll ever debug in IAM reduces to: which policies apply to this identity right now?

Reading a policy document

```
● ● ● s3-podinfo-readonly.json

{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "AllowReadPodinfoAssets",
    "Effect": "Allow",
    "Action": [
      "s3:GetObject",
      "s3:ListBucket"
    ],
    "Resource": [
      "arn:aws:s3:::podinfo-assets",
      "arn:aws:s3:::podinfo-assets/*"
    ],
    "Condition": {"IpAddress": {
      "aws:SourceIp": "202.51.0.0/16"
    }}
  ]
}
```

Effect

Allow or Deny. Explicit Deny always wins; default is implicit deny.

Action

service:Verb — s3:GetObject, ec2:*. Wildcards allowed, regretted later.

Resource

ARNs this applies to. Note bucket AND bucket/* are different things!

Condition

Optional guards: source IP, MFA present, time of day, tags.

Evaluation: explicit Deny > explicit Allow > implicit deny (default).

Machines don't get passwords: roles



Anti-pattern: create a user, download access keys, paste them into an EC2 instance / CI config / .env file. Keys leak — GitHub is full of them, and bots find each one in minutes.



EC2 instance

wears `role: podinfo-web`



STS issues temporary credentials
auto-rotated every few hours



S3 bucket

policy allows the role

EC2 / Lambda / EKS pods

Attach a role — the SDK finds credentials automatically.
Zero keys in code.

GitHub Actions

OIDC federation: the workflow assumes a role with a short-lived token. Same trick as our ECS deploy.

Humans (you)

SSO / assume-role into accounts. Long-lived access keys are last-resort.

The IAM hygiene checklist



Lock the root account

MFA on, access keys deleted, used only for the 5 tasks that truly require root.



Least privilege

Start from zero, add the specific actions a job needs. s3:* today is an incident tomorrow.



Policies → groups & roles

Never attach policies to individual users. People change jobs; groups don't.



Roles for every machine

EC2, Lambda, pods, pipelines — temporary credentials only. No keys in code, ever.



MFA for every human

Passwords leak. A \$0 authenticator app defeats the most common account takeover.



Audit with CloudTrail

Every API call is logged: who, what, when, from where. Review access, prune unused.

*If you remember one thing: humans get MFA, machines get roles, nobody gets s3:**

PART 4

S3

Simple Storage Service — an infinitely deep bucket of objects.
Eleven nines of durability, one API.



Object storage is a different animal



You need to store a million images, logs, backups, build artifacts. A disk (EBS) attaches to ONE instance and has a size. A shared filesystem is slow and fussy. Neither scales to “just keep everything, forever.”



Block — EBS

Raw disk sectors attached to one EC2 instance. The OS makes a filesystem on it.

GBs-TBs · single instance

Boot volumes, databases



File — EFS

A shared network filesystem (NFS). Many instances mount the same tree.

Elastic · multi-instance

Shared home dirs, CMS uploads



Object — S3

Whole files as immutable objects with a key, fetched over HTTP. No mounting, no sectors.

Unlimited · planet-wide

Images, logs, backups, artifacts, static sites



S3's deal: give me bytes + a key, I'll keep them with 99.999999999% durability and serve them anywhere via HTTPS. You give up: file locking, partial edits, POSIX. Objects are replaced, never edited.

Buckets, keys, and the folder illusion

```
● ● ● s3://podinfo-assets
```

```
s3://podinfo-assets/
```

```
logos/podinfo-dark.svg  
logos/podinfo-light.svg  
backups/2026-07-19/db.sql.gz  
backups/2026-07-20/db.sql.gz  
index.html
```

```
# 'logos/' is not a directory.  
# The KEY is the whole string:  
#   "backups/2026-07-20/db.sql.gz"  
# Flat namespace + prefix filtering  
#   = the folder ILLUSION.
```

Bucket

Top-level container. Name is GLOBALLY unique across all AWS accounts, lives in one region.

Object

The bytes + metadata. Up to 5 TB each. Addressable at <https://bucket.s3.region.amazonaws.com/key>

Key

The full path-looking string. Prefixes power listing, lifecycle rules & permissions.

Ships with, one checkbox away:



Versioning



Static website hosting



Encryption at rest



Presigned URLs



Event notifications

Event notifications → “when an object lands, trigger a Lambda” — remember this for Part 8.

Storage classes: the temperature scale

Hot**Standard**

Millisecond access, no retrieval fee

Active assets, websites

\$\$\$

Warm**Standard-IA**

Same speed, cheaper storage, per-GB retrieval fee

Backups you rarely read

\$\$

Cold**Glacier Instant / Flexible**

Cheap storage; Flexible takes minutes–hours to restore

Compliance archives

\$

Frozen**Glacier Deep Archive**

Cheapest bytes in AWS; ~12 h restore

“Keep 7 years, hope to never read”

¢



Don't choose manually — write a lifecycle rule: “after 30 days → IA, after 90 → Glacier, after 365 → delete.” Or enable Intelligent-Tiering and let S3 watch access patterns for you.

The everyday S3 muscle memory

```
● ● ● bash - aws-cli

$ aws s3 mb s3://sagyam-bootcamp-assets
make_bucket: sagyam-bootcamp-assets

$ aws s3 cp logo.svg s3://sagyam-bootcamp-assets/
$ aws s3 cp s3://sagyam-bootcamp-assets/logo.svg .

$ aws s3 sync ./site s3://sagyam-bootcamp-assets \
  --delete          # rsync for the cloud

$ aws s3 ls s3://sagyam-bootcamp-assets/backups/
2026-07-20 02:00:14    48213 db.sql.gz

$ aws s3 presign s3://sagyam-bootcamp-assets/report.pdf \
  --expires-in 3600
https://...?X-Amz-Signature=... # valid 1 hour
```

cp / sync

sync is incremental like rsync from the Linux module — the workhorse for deploys & backups.

s3 vs s3api

aws s3 = friendly porcelain. aws s3api = raw plumbing, every option (bucket policies, versions).

presign

Time-boxed HTTPS access to a private object. No IAM needed by the receiver.



Block Public Access stays ON unless you're hosting a website on purpose. “Accidentally public bucket” is the most cliché breach in cloud history.

PART 5

ECR

Elastic Container Registry — a private, IAM-guarded home for the images you've been building since the Docker module.



A registry that speaks IAM



Docker Hub free tier: rate limits, public by default, and a password shared across your team. Production needs a registry inside your cloud, guarded by the same IAM you use for everything else, close to where images are pulled.

Concept

You already know...

On ECR

Registry host

`docker.io` / `harbor.local`

`123456789.dkr.ecr.ap-south-1.amazonaws.com`

Repository

`sagyam/shortlink-api`

`shortlink-api` (one repo per image name)

Auth

`docker login` + password

`aws ecr get-login-password` → 12 h token via IAM

Who can pull?

`Public` / `robot accounts`

IAM policies — your EKS nodes' role, your CI role

Vulnerability scan

Trivy in Harbor

Scan-on-push (basic → Clair, enhanced → Inspector)

Free tier: 500 MB private storage / month. Pulls from the same region are fast and free of internet egress.

The push ritual

```
● ● ● bash - docker + aws-cli

# 1. one-time: create the repo
$ aws ecr create-repository --repository-name shortlink-api

# 2. login (token valid 12 h)
$ aws ecr get-login-password --region ap-south-1 \
  | docker login --username AWS --password-stdin \
    123456789.dkr.ecr.ap-south-1.amazonaws.com
Login Succeeded

# 3. tag with the full registry path
$ docker tag shortlink-api:1.4.2 \
  123456789.dkr.ecr.ap-south-1.amazonaws.com/shortlink-api:1.4.2

# 4. push
$ docker push 123456789.dkr.ecr.ap-south-1.../shortlink-api:1.4.2
1.4.2: digest: sha256:9f2a... size: 1573
```



Scan on push

Findings by CVE severity in the console & API. Fail the pipeline on HIGH/CRITICAL — same policy as our Trivy gate.



Lifecycle policies

“Keep last 10 tagged images, expire untagged after 7 days.” Registries grow forever without this — and you pay per GB.



In CI

GitHub Actions assumes an IAM role via OIDC, then steps 2–4 run headless. No stored passwords.



The tag IS the address: registry/repo:tag. Kubernetes manifests will point at exactly this string in Part 6.

PART 6

EKS

Elastic Kubernetes Service — your kubectl skills, AWS's control plane.
Everything from the K8s module, minus the 3 AM etcd pages.



Who runs what in EKS



Self-hosting K8s means babysitting etcd quorum, cert rotation, API-server HA, upgrades. That's a full-time job you don't want.

AWS manages — control plane

kube-apiserver

etcd (multi-AZ)

scheduler

controller-manager

\$0.10/hour flat · auto-scaled · patched & backed up for you



You manage — data plane

Managed node group (EC2)

Fargate profile (serverless pods)

Your pods, your instance types (spot!), your scaling — kubelet phones home to the managed API server



kubectl get pods works exactly as in the lab — the API server just lives at an AWS endpoint and authenticates you via IAM instead of a local kubeconfig cert.

Your K8s knowledge: a settlement map

Pods · Deployments · Services

unchanged

Same YAML, same kubectl apply

Ingress

upgraded

AWS Load Balancer Controller → an ALB per Ingress, real public IPs

PersistentVolumes

upgraded

EBS CSI driver → PVCs become real EBS volumes, AZ-aware

LoadBalancer Services

upgraded

type: LoadBalancer actually provisions an NLB (minikube could only pretend)

Cluster auth

changed

aws eks update-kubeconfig; identity = your IAM role, RBAC maps via access entries

Pod → AWS access

new

IRSA / Pod Identity: a pod assumes an IAM role — no keys in secrets

The mental shift: on EKS, Kubernetes objects can summon real AWS infrastructure. YAML in, load balancers out.

From zero to kubectl

```

● ● ● bash - eksctl + kubectl

$ eksctl create cluster \
  --name bootcamp \
  --region ap-south-1 \
  --nodegroup-name workers \
  --node-type t3.medium \
  --nodes 2 --spot
... 15-20 min: VPC, control plane, node group ...

$ aws eks update-kubeconfig --name bootcamp
Updated context arn:aws:eks:ap-south-1:::cluster/bootcamp

$ kubectl get nodes
NAME                                STATUS    VERSION
ip-192-168-12-34.ap-...             Ready    v1.33.1
ip-192-168-56-78.ap-...             Ready    v1.33.1

$ kubectl apply -f podinfo/          # old friends work

```

eksctl

The community CLI that turns one command into ~30 CloudFormation resources: VPC, subnets, IAM roles, cluster, nodes.

update-kubeconfig

Writes a kubeconfig entry whose 'password' is an IAM-signed token, minted fresh per kubectl call.

Cost care

Control plane \$0.10/h + nodes. ~\$2.40/day even when idle — create, lab, DELETE. eksctl delete cluster is homework.

Then the K8s module's Day 3–9 material replays unchanged — on real, internet-facing infrastructure.

PART 7

Telemetry

CloudWatch & X-Ray — metrics, logs and traces,
so 3 AM you can see what 3 PM you deployed.



The three pillars of observability

 It's 3 AM. Checkout is failing for some users. You have 40 containers on 6 nodes across 3 AZs. SSH-and-grep died with the single server.

Metrics

Numbers over time: CPU %, request rate, p99 latency, error count. Cheap to store, fast to alert on.

"Is something wrong?"

Logs

Timestamped event records with detail: stack traces, request bodies, audit trails. Expensive, rich.

"What exactly happened?"

Traces

One request's journey across services: web → auth → db → cache, with timing per hop.

"Where is it slow?"

 The debugging loop: a metric alarm wakes you → traces localize the slow/failing hop → logs of that service explain why. Each pillar answers the question the previous one raises.

CloudWatch: the built-in nervous system



Metrics

Every service reports automatically: EC2 CPU, Lambda invocations, S3 bucket size. Custom app metrics via API/agent. 1-min resolution standard.



Logs

Log groups → streams. The CW agent (or Fluent Bit on EKS) ships container stdout here. Logs Insights = a query language over them.



Alarms

Threshold or anomaly-detection on any metric → SNS → email / Slack / PagerDuty. Also auto-scaling triggers.



Dashboards

Pin metrics, logs queries & alarm states on one screen. Your NOC view.

```
● ● ● CloudWatch Logs Insights

fields @timestamp, @message
| filter status >= 500
| stats count(*) as errors by bin(5m), path
| sort errors desc | limit 10      # top failing endpoints, 5-min buckets
```

Tracing with X-Ray — and the wider map



260 ms inside one SQL query → the villain found in one glance.

Instrument

X-Ray SDK / OpenTelemetry in your app; a sidecar daemon batches spans to AWS.

Service map

Auto-drawn graph of your services with error % and latency per edge.

Sampling

Default: 1 req/s + 5% of the rest — insight without paying to trace everything.

Rosetta stone — the same jobs elsewhere:

Metrics + alerts	CloudWatch	Prometheus + Alertmanager
Logs	CloudWatch Logs	Loki · ELK stack
Traces	X-Ray	Jaeger · Tempo
Dashboards	CloudWatch Dashboards	Grafana

PART 8

Lambda

Serverless functions — no instances, no nodes, no patching.
Just code that runs when something happens.



No server, just a function



Task: resize each uploaded image. On EC2 that's a whole instance — patched, monitored, billed 24/7 — sitting 99% idle waiting for uploads. The task takes 2 seconds; the server exists all month.



The fine print that bites

```

● ● ● handler.py

import boto3, os
s3 = boto3.client("s3")    # init runs on COLD start only

def handler(event, context):
    # event: dict describing what happened
    rec = event["Records"][0]["s3"]
    key = rec["object"]["key"]
    ...resize, write back...
    return {"status": "ok", "key": key}

# IAM role attached to the function decides
# what boto3 is ALLOWED to do (Part 3!)

```

Great for

Glue between services · S3-triggered processing · cron jobs · webhooks · spiky APIs · ops automation

Wrong tool for

Long batch jobs · websockets & steady streams · latency-critical hot paths · local state

Cold starts

First request after idle pays init cost: ~100 ms–1 s+ (worse on JVM). Mitigate: small deps, provisioned concurrency for hot paths.

15-minute cap

Hard max per invocation. Longer work → chop it up (queues, Step Functions) or use a container on ECS/EKS.

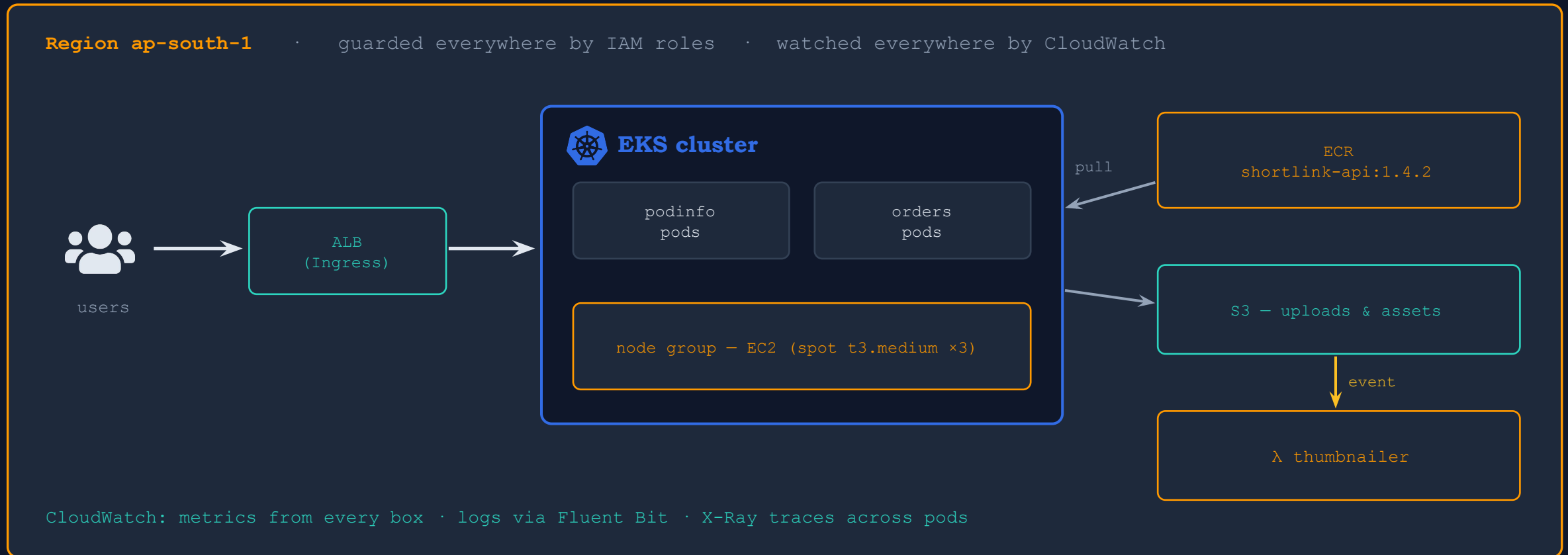
Stateless

The micro-VM may vanish between calls. State lives in S3 / DynamoDB / Redis — never in memory or /tmp long-term.

Memory = power

One dial, 128 MB–10 GB; CPU scales with it. Counterintuitive: more memory often FINISHES faster → cheaper.

One app, all eight parts



Every arrow is an IAM permission. Every box emits telemetry. Everything came from an API call — which means next step: writing this whole diagram as code (Terraform).

What you can now say with a straight face

Cloud 101

Self-service API infrastructure; IaaS→SaaS trade control for speed; you secure what you configure.

EC2

VMs by the second — AMI + type + EBS + SG + key pair. Spot for the brave, terminate what you forget.

IAM

Users, groups, roles, JSON policies. Humans get MFA, machines get roles, nobody gets s3:*

S3

Durable objects behind HTTP. Flat keys, storage classes by temperature, lifecycle rules do the moving.

ECR

Harbor's job with IAM auth — login token, tag with full path, push, scan, expire the old junk.

EKS

Managed control plane; your kubectl unchanged; Ingress/PVC now summon real AWS hardware.

Telemetry

Metrics ask if, logs ask what, traces ask where. CloudWatch is built in; OTel keeps you portable.

Lambda

Code that runs on events, scales itself, bills per ms — and disappears when idle.

Next stop:

Everything you clicked today, written as code.

Infrastructure as Code — because the console is for looking, not for building.



Labs: this week



Free tier: set a budget alarm first



blog.sagyamthapa.com.np